

FULL BUILD PLAN

AI Email Agent

A multi-user email assistant that drafts, reviews, and sends emails — built with Next.js, the OpenAI Agents SDK (running on Gemini), and Supabase, with the human always in control.

Framework: OpenAI Agents SDK → Gemini free API

Backend / Auth: Supabase (@supabase/ssr) + Sign in with Google

Frontend: Next.js 16 (App Router)

Level: First agentic AI project (beginner-friendly)

Estimated time: 3–4 weeks part-time (~55–75 hours)

• What's Inside

1 The Big Picture	What you're building & why it's "agentic"
2 Core Concepts	Agent vs Tool, in plain words
3 The Agents & Tools	Each one: what, why, and code
4 How It All Flows Together	The full request lifecycle
5 The Model — Gemini via the OpenAI SDK	Free API key setup + the one catch
6 Supabase — Database & Auth	Three-client architecture + Google login
7 Sending Email	nodemailer now, OAuth later
8 Build Order — Phase by Phase	The actual sequence to follow
9 Timeline & Checklist	Schedule + definition of done

1 The Big Picture

You're building a web app where a user signs in with Google, types a short description and a recipient's email, and an AI agent gathers context, writes a polished email, has a second agent review it, shows the result, and — only after the user approves — sends it. Because it's multi-user, each person connects their own Google account and only ever sees their own data.

The experience from the user's side:

1. User signs in with Google (one click handles both login and Gmail permission).
2. User writes: *"Follow up with sara@acme.com about the pricing we discussed, and propose a call next week."*
3. The agent works, then shows a complete, ready-to-send email.
4. User reads it. Likes it → **Approve & Send**. Doesn't → types *"make it shorter and more formal"* and the agent rewrites.
5. Once approved, the email is sent and saved to the database for future context.

Why this counts as "agentic AI"

Many beginner projects just send text to an AI and show the answer — that's a **wrapper**. What makes *this* agentic is that the AI is given a set of **tools** and **decides on its own** which to use for each task, plus a second agent that reviews the work. That decision-making is exactly what AI/agentic roles screen for.

Keep this in mind: The human always approves before anything sends. The agent is smart, but never sends on its own. This "human-in-the-loop" design is the correct, professional way to build anything that takes real-world actions.

2 Core Concepts (Plain Words)

Get these two words straight — almost everything here is one or the other.

A Tool

A **tool** is a normal function that *does one job* and returns a result: "look up this company," "search my old emails," "send this email." A tool has **no intelligence** — it just runs.

An Agent

An **agent** is the AI (an LLM) given instructions and a box of tools. It's where the **thinking** happens: it reads the task, decides *which* tools to call, reads their results, writes an answer, and can **hand off** to another agent.

The one sentence that matters: The agent *decides when* to call a tool; the tool *does the work* and hands data back. That back-and-forth is the whole idea of agentic AI.

	Tool	Agent
Is it smart?	No — plain code	Yes — an LLM that reasons
What it does	One specific job	Decides, calls tools, writes output
Example	<code>lookupCompany("acme.com")</code>	The "Writer" that drafts the email
Calls the other?	No	Yes — agents call tools & other agents

3 The Agents & Tools

You'll build **two agents** and a set of **tools**. Here's each — what it does, why it exists, and the code.

AGENT 1 The Writer Agent

What: Takes the description plus any context the tools gathered, and produces the email (subject + body).

Why: The main author. It decides which tools to call to gather what it needs before writing.

TYPESCRIPT — LIB/AGENTS/WRITER.TS

```
import { Agent } from '@openai/agents';
import { z } from 'zod';

const emailSchema = z.object({
  subject: z.string(),
  body: z.string(),
});

export const writerAgent = new Agent({
  name: 'EmailWriter',
  instructions: `You write professional emails from a short description.
Before writing, use your tools to gather context:
- look up the recipient's company if it's a business address
- check past emails to match the tone the user usually uses
- search previous threads if the task references an earlier conversation
Return a clear subject line and a well-formatted body.`,
  tools: [lookupCompany, lookupRecipientContext, searchPreviousThreads],
  outputType: emailSchema,
});
```

AGENT 2 The Reviewer Agent

What: Reads the Writer's draft and critiques it — tone, clarity, whether it matches the request. It sends notes back for a rewrite, or approves it for the human.

Why: A dedicated critic catches what the Writer misses, because the Writer is focused on *producing* and the Reviewer only on *judging*. Showing this "handoff" pattern is a strong signal to employers.

TYPESCRIPT — LIB/AGENTS/REVIEWER.TS

```
const reviewSchema = z.object({
  approved: z.boolean(),
  feedback: z.string(), // notes for the writer if not approved
});

export const reviewerAgent = new Agent({
  name: 'EmailReviewer',
  instructions: `You are a strict but fair email editor.
Check: is the tone right? Is the subject clear? Does it fully
answer the request? Any contradictions?
If good, set approved = true. Otherwise approved = false with
specific, short feedback.`,
  outputType: reviewSchema,
});
```

Why two agents, not one prompt saying "write then check"? A single prompt tends to be lenient on its own work. A separate reviewer with its own instructions is genuinely more critical — and it demonstrates multi-agent orchestration, a level above what most beginners show.

TOOL 1 lookupCompany(domain)

What: Takes the recipient's domain (e.g. `acme.com`), looks it up on the web, returns a short description.

Why: So the email references the recipient's real business instead of being generic. The Writer calls it only when the address looks like a company.

TYPESCRIPT — LIB/TOOLS/LOOKUPCOMPANY.TS

```
export const lookupCompany = tool({
  name: 'lookup_company',
  description: 'Look up what a company does from its email domain.',
  parameters: z.object({ domain: z.string() }),
  async execute({ domain }) {
    const info = await searchWeb(`what does ${domain} company do`);
    return info || 'No company info found.';
  },
});
```

Honest caveat: Only works for business domains. For a personal address like `sara.k.92@gmail.com` there's no company — the tool returns "nothing found" and the agent writes generically. Don't over-promise this feature.

TOOL 2 lookupRecipientContext(email)

What: Looks in *your own database* for past emails to this person; returns count, tone, recent subjects.

Why: So the agent writes to each person the way *you* usually do. Plain database code; the intelligence is the agent deciding to use it.

Important — where "past emails" come from: Your app only knows the emails *it* sent. Every approved send is saved to your Supabase database, and *that* is what these tools search — **not** the user's whole Gmail inbox. So the context tools start empty and get useful after a few emails have been sent through the app.

TYPESCRIPT — RUNS THROUGH THE SUPABASE ADMIN CLIENT (SEE §6)

```
export const lookupRecipientContext = tool({
  name: 'lookup_recipient_context',
  description: 'Find past emails to this recipient to match tone.',
  parameters: z.object({ email: z.string(), userId: z.string() }),
  async execute({ email, userId }) {
    const admin = createAdminClient();
    const { data } = await admin
      .from('emails')
      .select('subject, tone, sent_at')
      .eq('user_id', userId) // scope to this user
      .eq('recipient_email', email)
      .order('sent_at', { ascending: false })
      .limit(5);
    if (!data?.length) return { known: false };
    return { known: true, lastTone: data[0].tone,
      recentSubjects: data.map(r => r.subject) };
  },
});
```

TOOL 3 searchPreviousThreads(query)

What: Finds an earlier conversation *by meaning*, not exact words. Ask for "the pricing discussion" and it finds the thread even if that email said "cost" or "quote."

Why: So follow-ups reference what was said before. This is your **RAG** skill applied directly — the part that reads as "he really understands retrieval."

How it works

When you send an email, you also turn its text into a list of numbers (an **embedding**) that captures its meaning, and store it. To search, you embed the *query* the same way and find the stored emails whose numbers are **closest**. Supabase's Postgres supports this with the **pgvector** extension, called through a database function (RPC) — no raw driver needed.

TYPESCRIPT — SEMANTIC SEARCH VIA SUPABASE RPC

```

export const searchPreviousThreads = tool({
  name: 'search_previous_threads',
  description: 'Semantically search past emails by meaning.',
  parameters: z.object({ query: z.string(), userId: z.string() }),
  async execute({ query, userId }) {
    const embedding = await embed(query); // Gemini embeddings
    const admin = createAdminClient();
    const { data } = await admin.rpc('match_emails', {
      query_embedding: embedding,
      match_count: 3,
      p_user_id: userId, // manual scope; admin bypasses RLS
    });
    return data;
  },
});

```

Note on embedding size: If you use **Gemini** embeddings, set your `vector()` column to match **Gemini's** dimension — not OpenAI's 1536. Check the size your embedding model returns and use that number.

4 How It All Flows Together

The full journey of one request. Notice the Writer **chooses** which tools to call based on the task.

```
User (signed in): "Follow up with sara@acme.com about the pricing
we discussed, and propose a call next week."
|
▼
route identifies the user (Supabase server client reads cookie)
|
▼
WRITER AGENT reasons: "I need context"
├─ lookupCompany("acme.com")           → what Acme does
├─ lookupRecipientContext("sara@...")    → casual, 3 past emails
└─ searchPreviousThreads("pricing")      → finds the thread
|
▼
WRITER produces a draft using everything it gathered
|
▼
hands off to REVIEWER AGENT → checks tone, clarity, accuracy
├─ not good? → feedback back to Writer for a rewrite
└─ good?     → continue
|
▼
shown to the HUMAN → Approve · Tweak · Send
|
▼
on approve → sent, then saved to Supabase (scoped to user.id)
```

The human step never disappears. By the time the draft arrives, two agents and several tool calls have made it good — but the person still decides whether it sends.

5 The Model — Gemini via the OpenAI SDK

You're using the OpenAI Agents SDK as the **framework**, but running it on **Gemini's free API** to avoid paid keys. This works because Gemini offers an "OpenAI-compatible" endpoint — you just point the SDK at Gemini's URL with your Gemini key.

TYPESCRIPT — POINT THE AGENTS SDK AT GEMINI

```
import { setDefaultOpenAIClient, setOpenAIAPI, setTracingDisabled } from '@openai/agents';
import { OpenAI } from 'openai';

const gemini = new OpenAI({
  apiKey: process.env.GEMINI_API_KEY,
  baseURL: 'https://generativelanguage.googleapis.com/v1beta/openai/',
});

setDefaultOpenAIClient(gemini);
setOpenAIAPI('chat_completions');
setTracingDisabled(true); // tracing is OpenAI-only; turn it off with Gemini

// then, on each agent: model: 'gemini-2.5-flash' (or newer)
```

The one catch that affects THIS project: your Writer agent uses **tools + structured output together**. Older Gemini 2.x models can't do both at once. Use the **newest Gemini model** your free key supports (Gemini 3+ handles both). If tool calls misbehave with structured output, this is the first place to look.

Nice side benefit: because everything runs through the standard client, switching to a paid OpenAI key later means changing only these few lines — the rest of your code is untouched. You can honestly say: *"I built it provider-agnostic."*

Watch-outs with the free tier

- **Rate limits** — free Gemini caps requests per minute/day. Fine for building and demos; develop at a relaxed pace.
- **Embeddings** — Gemini's embedding model runs through the same endpoint (also free), so `searchPreviousThreads` stays free. Just match the vector column size (see §3).

6 Supabase — Database & Auth

Supabase is your backend: it stores emails/drafts, handles multi-user login, and runs pgvector search. It also solves auth elegantly — "Sign in with Google" handles both **login** and **Gmail permission** in one step.

Get credentials & install

In the Supabase Dashboard → **Project Settings** → **API**, collect three values: the **Project URL**, the **anon / publishable key** (safe for the browser, respects security rules), and the **service_role / secret key** (server-only, bypasses rules, never expose).

BASH

```
npm install @supabase/supabase-js @supabase/ssr
```

.ENV.LOCAL

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon / publishable key>
SUPABASE_SERVICE_ROLE_KEY=<service_role / secret key>
GEMINI_API_KEY=<your gemini key>
```

Security: The `NEXT_PUBLIC_` prefix means Next.js ships that variable to the browser. **Never** put it on the service_role key, or your admin key leaks to the client. Keep `.env.local` in `.gitignore`.

The three-client architecture

Supabase is used through three small clients, each with a distinct job:

File	Key	Runs in	Use for
<code>client.ts</code>	anon	Client Components	Browser reads/writes as the logged-in user
<code>server.ts</code>	anon	Server Components, Routes, Actions	Identifying the user (reads session cookie)
<code>admin.ts</code>	service_role	Server only	Agents, embeddings, pgvector — bypasses rules

TYPESCRIPT – LIB/SUPABASE/CLIENT.TS (BROWSER)

```
import { createBrowserClient } from '@supabase/ssr';
export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```

TYPESCRIPT – LIB/SUPABASE/SERVER.TS (SERVER)

```
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

// cookies() is async in Next.js 16, so this helper is async
export async function createClient() {
  const cookieStore = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() { return cookieStore.getAll(); },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options));
          } catch {} // refreshed by the proxy instead
        },
      },
    }
  );
}
```

TYPESCRIPT – LIB/SUPABASE/ADMIN.TS (SERVER ONLY)

```
import { createClient } from '@supabase/supabase-js';

// service_role key, bypasses rules. NEVER import into a Client Component.
export function createAdminClient() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );
}
```

Which client for queries? Acting as a user (respect their permissions) → `server.ts` . Acting as the system (agents, embeddings, pgvector) → `admin.ts` . Inside a "use client" component → `client.ts` . Most agent work uses `admin.ts` inside `app/api/*/route.ts` . You do **not** need a separate `db.ts` / raw Postgres driver — these three clients are your whole database layer.

Session refresh — the proxy (most-skipped step)

In Next.js 16, middleware is renamed **proxy**. It refreshes the auth token on every request so sessions don't silently expire.

TYPESCRIPT — LIB/SUPABASE/PROXY.TS

```
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';

export async function updateSession(request: NextRequest) {
  let response = NextResponse.next({ request });
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() { return request.cookies.getAll(); },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value));
          response = NextResponse.next({ request });
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options));
        },
      },
    },
  );
  await supabase.auth.getClaims(); // refreshes token if expired
  return response;
}
```

TYPESCRIPT — PROXY.TS (PROJECT ROOT)

```
import { type NextRequest } from 'next/server';
import { updateSession } from '@lib/supabase/proxy';

export async function proxy(request: NextRequest) {
  return await updateSession(request);
}
export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)
$).*(.*)'],
};
```

Sign in with Google

Because the app needs Gmail access anyway, let Google handle login and Gmail permission at once. Configure the Google provider in Dashboard → Authentication → Providers → Google, then:

TYPESCRIPT — APP/LOGIN/PAGE.TSX

```
'use client';
import { createClient } from '@lib/supabase/client';

async function signInWithGoogle() {
  const supabase = createClient();
  await supabase.auth.signInWithOAuth({
    provider: 'google',
    options: {
      scopes: 'https://www.googleapis.com/auth/gmail.send ' +
        'https://www.googleapis.com/auth/gmail.readonly',
      redirectTo: `${location.origin}/auth/callback`,
    },
  });
}
```

TYPESCRIPT — APP/AUTH/CALLBACK/ROUTE.TS

```
import { createClient } from '@lib/supabase/server';
import { NextResponse } from 'next/server';

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get('code');
  if (code) {
    const supabase = await createClient();
    await supabase.auth.exchangeCodeForSession(code);
  }
  return NextResponse.redirect(`${origin}/dashboard`);
}
```

Protect pages, and scope data with Row Level Security

Gate protected pages by checking for a user; if none, redirect to login. Then give every user-data table a `user_id` column and turn on Row Level Security so users only ever see their own rows.

SQL — SUPABASE SQL EDITOR

```
alter table emails enable row level security;

create policy "users manage own emails"
  on emails for all
  using (auth.uid() = user_id)
  with check (auth.uid() = user_id);
```

Key point: the `server.ts` /anon client obeys these rules automatically. The `admin.ts` client **bypasses** them — so whenever you write via admin, you must set `user_id` manually to keep each user's data separate.

How it runs together — one draft request

This pattern repeats across every route: **identify** with `server.ts`, then do the **work** with `admin.ts` scoped to `user.id`.

TYPESCRIPT — APP/API/DRAFTS/ROUTE.TS

```
import { createClient } from '@lib/supabase/server';
import { createAdminClient } from '@lib/supabase/admin';

export async function POST(req: Request) {
  // 1. WHO? — server client reads the cookie session
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getClaims();
  if (!user) return new Response('Unauthorized', { status: 401 });

  // 2. WORK — admin client runs agents + pgvector, scoped to user.id
  const admin = createAdminClient();
  const { data: context } = await admin.rpc('match_emails', {
    query_embedding: embedding,
    match_count: 5,
    p_user_id: user.id, // manual scope; admin bypasses RLS
  });

  // ... run Writer + Reviewer agents with that context ...
  await admin.from('drafts').insert({ user_id: user.id, content });
  return Response.json({ ok: true });
}
```

7 Sending Email — Start Simple, Upgrade Later

There are three ways to send, in rising order of effort. Build with the simplest first, then upgrade — the change is contained to one function; the rest of your app doesn't move.

Approach	Sends from	Effort	When
1. nodemailer + App Password	Only <i>your</i> account	Easiest	Early build & demo
2. nodemailer + OAuth2	Any user's account	Medium	Multi-user, keep nodemailer
3. Gmail API + OAuth	Any user's account	Fuller	Google-native path

Stage 1 — nodemailer + App Password (build with this)

An **App Password** is a 16-character password you generate in your Google account. nodemailer logs into Gmail's SMTP server with your email + that password. No consent screen, no tokens — quick to get working. The trade-off: it only sends from *your one* account, so it can't be multi-user.

TYPESCRIPT — LIB/MAILER.TS

```
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: process.env.MY_EMAIL,
    pass: process.env.APP_PASS, // 16-char app password
  },
});

export async function sendEmail(to, subject, body) {
  const info = await transporter.sendMail({
    from: process.env.MY_EMAIL, to, subject, text: body,
  });
  return info.messageId;
}
```

Guard your secrets: An App Password is a live key into your Gmail — anyone who has it can send as you. Never commit it, never paste it anywhere public, keep it in `.env.local`, and revoke any password that gets exposed (Google Account → Security → App passwords).

Stage 2 — the multi-user upgrade

To let *other* people send from *their* Gmail, you switch from an App Password to **OAuth** — the same "Sign in with Google" flow from §6 already collects the Gmail permission and tokens. Good news: **nodemailer**

supports OAuth2, so you can keep the library you already know and just feed it tokens instead of a password:

TYPESCRIPT – NODemailer WITH OAUTH2 (MULTI-USER)

```
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    type: 'OAuth2',
    user: userEmail, // the connected user's gmail
    clientId: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    refreshToken: userRefreshToken, // from the Google sign-in flow
  },
});
```

Why OAuth for a real app: it's how a user grants permission to use *their* Gmail without ever giving you their password — like a hotel key card that opens only their room and can be revoked anytime. That's exactly why the multi-user version needs it, and why an App Password (tied to your single account) can't get you there.

8 Build Order — Phase by Phase

Don't build everything at once. Each phase gives you something that works before the next layer. This ordering is the difference between finishing and getting stuck.

Phase 1 — Skeleton that works

User types description + email → one agent (on Gemini) writes it → show it on screen. No tools, no database, no login yet. Prove the round-trip first.

TYPESCRIPT — THE ORCHESTRATOR THAT RUNS THE TWO AGENTS

```
import { run } from '@openai/agents';
import { writerAgent } from './agents/writer';
import { reviewerAgent } from './agents/reviewer';

export async function makeEmail(description: string) {
  let draft = (await run(writerAgent, description)).finalOutput;

  for (let i = 0; i < 2; i++) { // up to 2 rewrites
    const review = (await run(reviewerAgent,
      `Draft: ${JSON.stringify(draft)}\nRequest: ${description}`
    )).finalOutput;
    if (review.approved) break;
    draft = (await run(writerAgent,
      `${description}\nPrevious: ${JSON.stringify(draft)}\nFix: ${review.feedback}`
    )).finalOutput;
  }
  return draft; // { subject, body }
}
```

Phase 2 — User revision loop

Add the "tweak it" box. When the user types feedback like *"make it shorter,"* pass the old draft + feedback back to the Writer. Same idea as the Reviewer loop, but the human gives the notes.

Phase 3 — Sending (Stage 1)

Wire in **nodemailer + App Password** so an approved email actually sends from your account. Simple, and it unblocks the whole flow.

Phase 4 — Supabase: database + login

Set up the three clients, the proxy, Sign in with Google, and RLS (§6). Save every sent email to the `emails` table scoped to `user_id`.

Phase 5 — Context tools

Add `lookupRecipientContext` (easy Supabase query, instant payoff), then `searchPreviousThreads` with pgvector via the `match_emails` RPC (the résumé-worthy RAG piece).

Phase 6 — Polish & the multi-user upgrade

Add `lookupCompany` (web search). Upgrade sending to **OAuth** (Stage 2) so other users can connect their own Gmail. Tidy the UI, add error handling, write a strong README.

Golden rule: A working three-feature version beats a half-broken six-feature one every time. Get Phases 1–4 solid before the fancy parts.

9 Timeline & Checklist

Realistic estimate for a first agentic project, part-time (~15–20 hrs/week). Full-time, compress to about 8–10 focused days.

Week	What you build	Phases	Hours
Week 1	Setup, Gemini-on-SDK wiring, Writer + Reviewer drafting on screen, user revision loop.	1–2	15–18
Week 2	nodemailer + App Password sending. Then Supabase: three clients, proxy, Google login, RLS.	3–4	16–20
Week 3	Save sent emails, <code>lookupRecipientContext</code> , then pgvector + <code>match_emails</code> semantic search (RAG).	5	15–20
Week 4	Company lookup, OAuth multi-user upgrade, UI polish, error handling, README.	6	10–15

Total: ~55–75 hours over 3–4 weeks part-time. The trickiest stretch is the Supabase auth wiring in Week 2 — more configuration than code — so don't be discouraged if the proxy/login step takes an evening to click.

If you're short on time: a polished version of Phases 1–4 (draft → review → approve → send → save, with login) is a genuinely solid portfolio piece on its own — about 2 weeks. The RAG search in Week 3 is what takes it from good to impressive.

Definition of done

- ☐ User signs in with Google; each user sees only their own data (RLS on).
- ☐ Two agents: a Writer that drafts and a Reviewer that critiques before the human sees it.
- ☐ User can type feedback and the agent rewrites.
- ☐ Nothing sends until the user clicks Approve & Send.
- ☐ Approved email sends (App Password first; OAuth for multi-user) and is saved to Supabase.
- ☐ At least one context tool works; ideally semantic thread search via pgvector.

What makes it impressive to employers

- **Real tools the agent chooses to call** — separates "an agent" from "an API wrapper." Show the agent deciding, not you hardcoding.
- **Writer → Reviewer handoff** — demonstrates multi-agent orchestration.
- **RAG semantic search** — proves you understand retrieval, not keyword matching.

- **Provider-agnostic + multi-user auth** — Gemini-swappable-for-OpenAI, plus real Supabase RLS, reads as production awareness.
- **A README explaining the *why*** — the architectural reasoning often impresses more than the code.

One caution: don't add every possible feature. A focused project doing a few agentic things well beats a sprawling one doing many shallowly. Build the two agents, the tools, one memory feature, and multi-user auth properly — then explain your choices.

— End of plan · Build it phase by phase, and you'll have a portfolio piece worth talking about in interviews —